SOFTWARE ENGINEERING DEPARTMENT SPECIFICATION

Software Requirements Specification (SRS)

Project Title: Movie Information Portal

System Title	Movie Information Portal
Approved Tech Stack	HTML5, CSS3, Bootstrap 5 / Tailwind CSS, JS ES6+, jQuery
Data Architecture	Local JavaScript Static Dataset (js/movies.js) — Zero Network/Storage Dependency

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the structural, functional, and non-functional requirements for the Movie Information Portal. Formulated in compliance with IEEE Std 830-1998 standards, this specification serves as the definitive engineering blueprint for developing an interactive, zero-dependency client-side web portal. The application is built exclusively using core web technologies—HTML5, CSS3, Bootstrap 5 / Tailwind CSS, JavaScript ES6+, and jQuery—and operates seamlessly in completely offline environments using a local JavaScript dataset.

1.2 Scope

The Movie Information Portal delivers a responsive, high-performance web presentation layer. The system scope encompasses client-side operations including:

- **Static Catalog Sourcing:** Reading and parsing movie metadata from an embedded JavaScript dataset array (js/movies.js).
- **Real-time Keyword Search:** Filtering displayed cards instantly based on user keyup input events across titles, directors, and descriptions.
- **Multi-Category Taxonomy Filtering:** Filtering movie items by genre, release year, and rating baselines using native JavaScript array methods.
- **Multi-Criteria Sorting:** Re-ordering array views dynamically by rating, year, or title in real time.
- **Dynamic Modal & Detail Views:** Injecting rich metadata, cast listings, and synopses dynamically into the DOM using jQuery.
- **In-Memory Session Watchlist:** Managing user favorite selections within runtime JavaScript array variables (without LocalStorage or SessionStorage).
- **Interactive Form Validation:** Intercepting contact inquiries and performing client-side regex input validation via jQuery.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Engineering Definition
SRS	Software Requirements Specification; a formal document outlining system capabilities and constraints.
DOM	Document Object Model; the structural standard for HTML elements manipulated at runtime by JavaScript and jQuery.
jQuery	A fast, lightweight JavaScript library used strictly for client DOM manipulation, event handling, animations, and validation.
In-Memory State	Temporary runtime data maintained within active JavaScript variables during a tab session (cleared on refresh).
Static Dataset	Pre-compiled JavaScript array of objects (js/movies.js) loaded synchronously into memory.

2. Overall Description

2.1 Product Perspective

The Movie Information Portal functions as a self-contained, client-side rich web application. Operating entirely inside the user's browser, the application relies on no external REST APIs, no third-party cloud services (such as TMDB or OMDb), and no persistent browser database storage (such as LocalStorage or SessionStorage). All movie entity records are read synchronously from js/movies.js upon initial document load, delivering zero network latency and 100% offline reliability.

2.2 Operating Stack & Technical Constraints

- **Approved Technologies:** HTML5, CSS3, Bootstrap 5 / Tailwind CSS, JavaScript ES6+, jQuery v3.6+.

- **Strictly Prohibited Technologies:** React, Angular, Vue, REST APIs, OMDb API, TMDB API, AJAX requests, Fetch API, Axios, LocalStorage, SessionStorage, IndexedDB, dynamic server languages (PHP, Node.js, Python).
- **jQuery Scope Limitations:** jQuery usage is strictly confined to DOM traversal, event binding, visual animation toggles (fadeIn, fadeOut, slideToggle), dynamic HTML string rendering, and client form validation. jQuery AJAX is prohibited.

3. Dataset Schema & In-Memory State Specifications

All application movie records are structured as an array of JavaScript objects defined inside js/movies.js. The schema baseline is strictly specified below:

3.1 Master Movie Object Schema (js/movies.js)

```
/**
 * Master Movie Dataset Structure
 * File: js/movies.js
 */
const movieDataSet = [
  {
    id: 101,
    title: "Inception",
    genre: "Science Fiction",
    language: "English",
    year: 2010,
    duration: "148 min",
    rating: 8.8,
    director: "Christopher Nolan",
    cast: ["Leonardo DiCaprio", "Joseph Gordon-Levitt", "Elliot Page", "Tom Hardy"],
    poster: "images/posters/inception.jpg",
    description: "A thief who steals corporate secrets through dream-sharing technology is
given the inverse task of planting an idea into a CEO's mind."
  },
  {
    id: 102,
    title: "The Dark Knight",
    genre: "Action",
    language: "English",
    year: 2008,
    duration: "152 min",
    rating: 9.0,
    director: "Christopher Nolan",
    cast: ["Christian Bale", "Heath Ledger", "Aaron Eckhart", "Michael Caine"],
    poster: "images/posters/dark_knight.jpg",
    description: "When the menace known as the Joker wreaks havoc on Gotham, Batman must accept
one of the greatest psychological tests of his ability to fight injustice."
  }
];
```

3.2 In-Memory Runtime Watchlist Array Structure

```
/**
 * Runtime In-Memory Favorites Collection (No LocalStorage)
 * File: js/app.js
 */
let runtimeWatchlist = []; // Contains movie IDs added during active session

function toggleFavorite(movieId) {
  const index = runtimeWatchlist.indexOf(movieId);
  if (index === -1) {
    runtimeWatchlist.push(movieId);
  } else {
    runtimeWatchlist.splice(index, 1);
  }
  updateWatchlistUI();
}
```

4. Functional Requirements

FR-1: Home Page & Feature Highlights Module

Description: Provides the primary portal landing view featuring spotlight hero displays, top-rated movies, quick genre categories, and dynamically calculated dataset summary metrics.

Functional Requirements Specifications:

- The system shall dynamically calculate total catalog count, average rating, and total genres from movies.js using JavaScript array reduce.
- The system shall render a hero carousel displaying top-rated movie cards upon document ready event.

FR-2: Navigation & Layout Management System

Description: Delivers a persistent, responsive navigation bar ensuring uniform layout access across all pages.

Functional Requirements Specifications:

- The navigation header shall include working links to Home, Movie Catalogue, Search, About, and Watchlist.
- On viewports below 768px, the navigation bar shall collapse into a hamburger menu using Bootstrap 5 collapse component / jQuery handlers.

FR-3: Movie Catalogue Grid & Pagination

Description: Renders movie cards into responsive grid structures with client-side pagination.

Functional Requirements Specifications:

- The system shall iterate over movieDataSet and dynamically append HTML card templates into the catalog grid container using jQuery.
- The catalogue shall implement array slicing to paginate results, displaying a maximum of 8 items per page with dynamic page controls.

FR-4: Movie Details & Modal View Subsystem

Description: Renders detailed metadata overlays for selected movie items.

Functional Requirements Specifications:

- Upon user click of 'View Details', the system shall query movieDataSet by numerical ID using Array.find().
- The detail modal shall display synopsis, director, complete cast tags, duration, rating badges, runtime favorite toggle, and a print trigger.

FR-5: Real-time Keyword Search Engine

Description: Provides instant client-side keyword filtering across movie records as the user types.

Functional Requirements Specifications:

- The search engine shall bind jQuery keyup and input listeners to the search field.
- Search matching shall evaluate case-insensitive string checks against title, director, and description properties.
- If no matching records exist, the system shall display a clean empty-state UI notification using jQuery fadeIn().

FR-6: Multi-Category & Taxonomy Filter

Description: Enables multi-category exploration through genre selector controls.

Functional Requirements Specifications:

- The taxonomy module shall dynamically derive unique genre categories present across movies.js on page initialization.
- Clicking a category filter button shall filter the active array view using Array.filter() and re-render the catalog grid.

FR-7: Multi-Attribute Sorting Controller

Description: Re-orders the display sequence of current array items based on selected criteria.

Functional Requirements Specifications:

- The sorting module shall provide dropdown choices: 'Rating (High to Low)', 'Year (Newest First)', 'Year (Oldest First)', and 'Title (A-Z)'.
- The system shall execute in-memory sorting via Array.sort() without mutating the master dataset array.

FR-8: Responsive Grid & Fluid Layout Controller

Description: Ensures visual layout stability across desktop, tablet, and mobile displays.

Functional Requirements Specifications:

- The layout shall conform strictly to Bootstrap 5 / Tailwind 12-column grid breakpoints (1 col on mobile, 2 cols on tablet, 3-4 cols on desktop).
- All image elements shall enforce img-fluid responsive styles to prevent overflow errors.

FR-9: Client-Side Contact & Feedback Form Validation

Description: Manages form input validation for user inquiries on the About page using jQuery.

Functional Requirements Specifications:

- The validation logic shall intercept form submit events via jQuery submit listener and prevent browser page reloads.
- Fields (Full Name, Email, Message) shall be validated against regex pattern checks for email formatting and minimum length.
- Upon validation success, the system shall show a success banner via jQuery slideDown() and reset form inputs.

FR-10: About Page & Architecture Showcase

Description: Outlines portal objectives, technology stack tags, dataset schema explanations, and interactive accordions.

Functional Requirements Specifications:

- The About page shall render technical documentation breakdowns and dynamic interactive accordions using jQuery toggle animations.

5. Non-Functional Requirements

NFR Category	Standard Engineering Specification
NFR-1: Performance	Initial document load shall complete under 300ms. Client-side search and filtering operations shall re-render DOM grids within 30ms of user input.
NFR-2: Reliability	Zero runtime console errors during user interactions. All invalid searches shall decay gracefully into standardized empty-state feedback alerts.
NFR-3: Maintainability	Strict separation of concerns: HTML markup structure, CSS aesthetics, jQuery behavior, and movies.js data definitions.
NFR-4: Scalability	The client architecture shall support scaling movies.js up to 1,000 array entries without degradation of filtering speed.
NFR-5: Usability	Clean UI visual hierarchy adhering to contrast ratio guidelines (WCAG 4.5:1 minimum) with responsive hover visual feedback states.
NFR-6: Security	All dynamic dynamic HTML string insertions via jQuery shall sanitize text inputs (e.g., using .text()) to prevent DOM Cross-Site Scripting (XSS).
NFR-7: Availability	As a fully self-contained local web client, system operational availability shall achieve 100% offline uptime.
NFR-8: Compatibility	Cross-browser compatible across Chrome, Firefox, Safari, Edge, and mobile viewports (iOS Safari, Android Chrome).
NFR-9: Accessibility	HTML document markup shall utilize semantic structural elements (<header>, <nav>, <main>, <section>, <footer>) and ARIA attributes.

6. Suggested Folder Structure

The project directory workspace must adhere to the exact clean structure outlined below:

```
movie-information-portal/
├── index.html            # Main Portal Landing Page & Spotlight Carousel
├── movies.html           # Full Movie Catalogue with Filters & Sorting
├── details.html          # Individual Movie Metadata Detail View
├── about.html            # System Architecture, Tech Stack & Contact Form
│
├── css/
│   ├── bootstrap.min.css # Bootstrap 5 Core CSS Framework (or tailwind.min.css)
│   └── styles.css        # Custom Styling, Dark Theme Variables & Visual Effects
│
├── js/
│   ├── jquery.min.js     # jQuery Library v3.6.+
│   ├── bootstrap.bundle.js# Bootstrap 5 Component JS
│   ├── movies.js         # Master JavaScript Array Dataset
│   ├── app.js            # Catalog Controller, Filtering, Sorting & Modal Logic
│   └── validation.js     # jQuery Form Validation & Interactive Effects
│
└── images/
    ├── posters/          # High-resolution film poster graphic assets
    └── assets/           # UI Icons, Brand Assets, and Hero Banners
```

7. Core JavaScript & jQuery Implementation Guide

The application logic separates data definitions from DOM manipulation routines across the following core scripts:

Step 1: Include Scripts Synchronously in HTML

Load jQuery, master dataset movies.js, and app logic before closing body tag:

```
<!-- HTML Script Inclusions -->
<script src="js/jquery.min.js"></script>
<script src="js/bootstrap.bundle.min.js"></script>
<script src="js/movies.js"></script>
<script src="js/app.js"></script>
<script src="js/validation.js"></script>
```

Step 2: Dynamic Card Rendering Logic (app.js)

Function iterating over dataset and injecting styled Bootstrap cards into DOM:

```
// Function to render movie cards using jQuery
function renderMovieGrid(moviesArray) {
  const $container = $('#movieGridContainer');
  $container.empty();

  if (moviesArray.length === 0) {
    $('#emptyStateNotice').fadeIn();
    return;
  }
  $('#emptyStateNotice').hide();

  moviesArray.forEach(movie => {
    const cardHtml = `
      <div class="col-md-4 col-lg-3 mb-4">
        <div class="card h-100 movie-card shadow-sm">
          <img src="${movie.poster}" class="card-img-top" alt="${movie.title}">
          <div class="card-body">
            <span class="badge bg-primary float-end">★ ${movie.rating}</span>
            <h5 class="card-title">${movie.title}</h5>
            <p class="text-muted mb-1">${movie.genre} | ${movie.year}</p>
            <button class="btn btn-outline-primary btn-sm w-100 btn-details" data-
id="${movie.id}">
              View Details
            </button>
          </div>
        </div>
      </div>`;
    $container.append(cardHtml);
  });
}
```

Step 3: Real-Time Search & Filtering Handler

Binding keyup listener to search input field:

```
// jQuery Keyup Event Listener for Live Search
$('#searchInput').on('keyup input', function() {
  const query = $(this).val().toLowerCase().trim();
  const filtered = movieDataSet.filter(movie =>
    movie.title.toLowerCase().includes(query) ||
    movie.director.toLowerCase().includes(query) ||
    movie.genre.toLowerCase().includes(query)
  );
  renderMovieGrid(filtered);
});
```

8. UI Screens Matrix

Screen / View	Key UI Components	Functional Behavior & Interactive Features
Home Screen	Hero Carousel, Top Rated Row, Category Quick Links, Summary Metric Counter	Presents spotlight movies via animated slide carousels. Dynamically computes catalog totals via JS reduce.
Movie Listing Screen	Genre Filter Sidebar, Search Field, Sorting Dropdown, Movie Card Grid, Pagination Bar	Main catalog interface displaying card grid. Interactively updates displayed cards upon search typing or filter toggles.
Movie Details Modal / Page	Poster Showcase, Metadata Badges, Synopsis Text, Cast Tag Pills, Watchlist Toggle Button	Renders exhaustive metadata for selected film item. Offers controls to add title to runtime watchlist.
Search Screen / Overlay	Real-Time Keyword Input, Clear Button, Result Counter Label, Grid Container	Live search view evaluating title keywords and director strings. Shows empty state notice if no match found.
About Screen	Tech Stack Badges, Schema Breakdown Box, Developer Accordions, Contact Validation Form	Displays project technical context and provides client-validated user feedback submission form.
Error / Empty State View	Illustration Graphic, Clear Filters Button, Friendly Alert Notice	Triggers automatically when search or filter combinations return zero matching items from movies.js.

9. Advanced Bonus Features Framework

Advanced development benchmarks implemented using pure client-side code:

- **Dark Theme Mode Switcher:** Client-side runtime theme toggler using CSS custom variables and jQuery class toggles to switch between dark and light visual themes.
- **Live Search Suggestions Dropdown:** Predictive typeahead dropdown box appearing under search bar showing matching movie titles in real time as user types.
- **Advanced Multi-Criteria Filtering:** Combined search logic allowing simultaneous filtering by Genre, Rating Threshold (e.g., Rating > 8.0), and Year Range.
- **In-Memory Runtime Watchlist / Favorites:** Ability to mark titles as 'Favorites' saved in an active JS array state during the session (without LocalStorage).
- **Movie Spec-Sheet Print Generator:** One-click printable spec-sheet generator formatting movie details cleanly via custom CSS @media print rules.
- **Side-by-Side Movie Comparison Matrix:** Interactive modal permitting selection of two movies to compare metrics (Rating, Runtime, Director, Year, Genre) side-by-side.

10. Academic & Industrial Evaluation Rubric

Evaluation Category	Weightage	Key Assessment Parameters & Technical Focus
HTML5 Structural Quality	10 Marks	Proper semantic tag usage, clean DOM structure, accessibility attributes (ARIA roles).
CSS3 & Styling Aesthetics	10 Marks	Custom styling, CSS variable architecture, typography, clean UI presentation.
Bootstrap 5 / Tailwind Grid	10 Marks	Responsive layout grid, component utilization (modals, cards, navbar), utility class usage.
JavaScript ES6 Fundamentals	15 Marks	Dataset schema structure, array methods (filter, sort, map, reduce), clean state logic.
jQuery Operations	15 Marks	Effective DOM manipulation, event binding, animations (fadeIn, slideToggle), zero AJAX usage.
Responsive Layout Design	10 Marks	Seamless rendering across mobile, tablet, and desktop displays; operational navigation drawer.

Evaluation Category	Weightage	Key Assessment Parameters & Technical Focus
UI/UX Aesthetics	10 Marks	Visual balance, intuitive navigation cues, active state indicators, hover visual transitions.
Code Quality & Architecture	10 Marks	Modularity, separation of concerns, structured code comments, DRY principles, zero console errors.
Documentation Quality	5 Marks	Completeness of SRS document, inline code documentation, accurate folder structure.
Technical Viva / Defense	5 Marks	Demonstrated understanding of DOM events, array processing, responsive layout principles.
TOTAL BASELINE SCORE	100 Marks	Comprehensive Evaluation Threshold for Highest Academic & Industrial Distinction

11. 12-Week Detailed Execution & Milestone Plan

Wk	Date Range	Topic / Phase	Activities & Concepts	Outcome / Deliverables
1	27/07/2026 To 01/08/2026	Requirement Analysis	Define project scope, finalize IEEE SRS document layout, set up repository	Approved SRS Document
2	03/08/2026 To 08/08/2026	Architecture & Data Design	Design local array schema in movies.js, structure directory layout, gather poster assets	movies.js Schema File
3	10/08/2026 To 15/08/2026	Static HTML Wireframing	Build semantic HTML wireframe templates for index, catalog, details, and about pages	Static HTML Wireframes
4	17/08/2026 To 22/08/2026	Bootstrap / Tailwind Layout	Implement responsive navigation bar, hero banner, footer, and fluid grid break points	Responsive Layout Shell
--	**24/08/2026 To 29/08/2026**	**Phase – 1 Evaluation**	**Formal progress review of static HTML/CSS/bootstrap structural wireframes and design alignment**	**Phase-1 Sign-off**
5	24/08/2026 To 29/08/2026	Custom CSS & Aesthetics	Develop custom styling, dark theme CSS variables, card hover transitions, and badge aesthetics	Polished Layout Theme
6	07/09/2026 To 12/09/2026	Dataset Logic & Rendering	Populate movies.js dataset, write card template generator functions in app.js	Dynamic Card Generator
7	14/09/2026 To 19/09/2026	jQuery DOM & Modals	Bind click handlers, modal dynamic popups, and dynamic catalog DOM injection routines	Interactive Catalog View

Wk	Date Range	Topic / Phase	Activities & Concepts	Outcome / Deliverables
8	21/09/2026 To 26/09/2026	Real-Time Search Engine	Build live search input listeners and category filter triggers using JS array filter()	Live Search & Filter Engine
--	**28/09/2026 To 03/10/2026**	**Phase – 2 Evaluation**	**Core evaluation assessing dataset rendering, card generation, and jQuery DOM binding**	**Phase-2 Sign-off**
9	28/09/2026 To 03/10/2026	Sorting & Pagination	Implement multi-attribute array sorting (Array.sort) and client-side page slicing	Sorting & Pagination Module
10	05/10/2026 To 10/10/2026	Form Validation & UI FX	Implement jQuery form validation, accordion toggles, dark theme switcher, animations	Validated Interactive Form
11	12/10/2026 To 17/10/2026	Testing & Cross-Browser QA	Conduct cross-browser layout audits, mobile view tests, accessibility checks, performance audit	QA Test Execution Report
12	19/10/2026 To 24/10/2026	Documentation & Prep	Finalize code comments, inline documentation, prepare demo presentation and viva defense	Final Project Package
--	**26/10/2026 To 07/11/2026**	**Phase – 3 Final Evaluation**	**Final project presentation, code defense viva, live software demonstration, and code submission**	**Final Grade Sign-off**